

## Часть II

Вычисления на логарифмической памяти. Класс  $L$  и  $NL$ . Теорема Рейнгольда.

Логарифмическая сводимость. Примеры  $NL$ -полных языков. Теорема Иммермана-Селепенки.

Зам Сейчас важно, что теперь у МТ будут отдельно входные ленты (с которых можно только читать), рабочие ленты и выходные (на которых можно писать слева направо и ничего более). Затраченную память мы далее будем считать только на рабочих лентах, то есть можно будет читать слова любой длины с входных лент и писать произвольно на выходные, если не наложим дополнительных условий, и это не будет влиять на использующую память.

$$\text{Опр } L = PSPACE(\log n)$$

$$\text{Опр } NL = NSPACE(\log n)$$

Теорема (Рейнгольда,  $\delta/g$ )  $\supset$   $GRAPH = \{G; s; t\} \mid \text{в неориентированном графе } G \text{ есть путь из } s \text{ в } t\} \in L$ .

Примеры языков класса  $L$ :

$$\textcircled{1} LE = \{(x; y) \mid x \leq y\}$$

□ Будем хранить 2 числа - номер текущего символа чисел  $x; y$ . Брать соответствующие и сравнивать. Каждый счётчик в двоичной системе займёт  $\log_2 \lceil \max\{x, y\} \rceil$  памяти  $\Rightarrow$   
 $\Rightarrow LE \in L$  ■

$$\textcircled{2} ADD_E = \{(x; y; z) \mid x + y = z\}$$

□ Аналогично храним 2 счётчика + сохраняем, есть ли 1 к следующему разряду и

сравниваем с  $x$ , то есть просто складываем столбиком ■

③  $MULT = \{ (x; y; z) \mid xy = z \}$

□ Реализуем умножение столбиком, но тут чуть больше надо запомнить цифр ■

④  $TREE = \{ G \mid \text{неориентированный граф } G \text{ является деревом} \}$

□  $G$ -дерево  $\Leftrightarrow G$ -связный и в нём  $n-1$  рёбро, где  $n$  - количество вершин.

Можно посчитать сначала количество рёбер. Потом перебирать вершины от 2 до  $n$  и смотреть, есть ли путь от 1 до текущей вершины. Это можно сделать за

логарифм доп. памяти по теореме Рейнгольда.  $\Rightarrow TREE \in L$  ■

⑤ Язык правильных скобочных последовательностей

□ Для константного числа видов скобок можно модифицировать скобочный баланс, добавив последующие проверки.

---

Опр Функция  $f$  вычислима на логарифмической памяти  $\Leftrightarrow$  язык

$D_f = \{ (x; k) \mid |f(x)| \leq k \} \in L$  и  $E_f = \{ (x; i) \mid f(x) = i \} \in L$  и  $\max k$ , при котором  $(x; k) \in D_f$  ограничено полиномом от  $|x|$ .

Утв (б/г) композиция функций, вычисляемых на логарифмической памяти, вычислима на логарифмической памяти.

Опр Язык  $A$  логарифмически сводится к языку  $B$ , если  $f$  вычисляемая на логарифмической памяти функция  $f$  такая, что  $x \in A \Leftrightarrow f(x) \in B$ . Обозначение  $A \leq_L B$ .

Свойства логарифмической сводимости:

- 1)  $\forall A \hookrightarrow A \leq_L A$
- 2)  $\forall A, B, C: A \leq_L B, B \leq_L C \Rightarrow A \leq_L C$
- 3)  $\forall A \in L, \forall B \in \emptyset \cup \{0, 1\}^* \hookrightarrow A \leq_L B$
- 4)  $\forall A, B: B \in L: A \leq_L B \Rightarrow A \in L$
- 5)  $\forall A, B: B \in NL: A \leq_L B \Rightarrow A \in NL$

Опр Язык  $B$  называется  $NL$ -трудным, если  $\forall A \in NL \hookrightarrow A \leq_L B$

Опр Язык  $B$  называется  $NL$ -полным, если  $B \in NL$  и  $B$  -  $NL$ -трудный. Обозначение:  $B \in NLC$ .

Опр (сертификатное определение  $NL$ )

$A \in NL \Leftrightarrow \exists V \vdash MT: V(x; s)$  (то есть принимает 2 аргумента и выполняет условие:

- 1)  $V$  - ДМТ, которая работает на логарифмической памяти
- 2) Слово  $x$  находится на своей входной ленте, и его можно читать без ограничений.
- 3) Слово  $s$  находится на второй входной ленте, и его можно читать всего один раз слева направо.
- 4)  $x \in A \Leftrightarrow \exists s V(x; s) = 1$

Примеры  $NL$ -полных языков:

①  $PATH = \{ \langle G; s; t \rangle \mid \text{в ориентированном графе } G \text{ есть путь из } s \text{ в } t \}$

Теорема (с/г)  $PATH \in NLC$

②  $SCONNECTED = \{G \mid \text{ориентированный граф } G \text{ сильно связный}\}$ .

Теорема  $SCONNECTED \in NLC$

□ 1) Сначала докажем, что  $SCONNECTED \in NL$ . Надо проверить, что есть путь от  $\forall$  вершин  $u$  до  $\forall$  вершин  $v$ . Сделаем перебор по парам  $(u, v)$ . Т.к.  $DPATH \in NL$ , то можно за логарифм по памяти на НЛТ проверить, есть ли путь из  $u$  до  $v$ . Если нет, то выводим 0, если да, то продолжаем перебор. После перебора выводим 1, т.к. иначе бы уже было 0. Итого, мы на каждом шаге цикла используем логарифмическую память, а с каждым новым шагом эта память может быть периспользована  $\Rightarrow$

$\Rightarrow SCONNECTED \in NL$

2) Теперь докажем, что  $SCONNECTED$  —  $NL$ -трудный. Для этого хотим  $DPATH \leq L SCONNECTED$ . Надо построить функцию  $f$ , принимающую на вход  $(G; s; t)$ , возвращающую граф  $G'$ , вычисляемую на логарифмической памяти такую, что  $(G; s; t) \in DPATH \Leftrightarrow G' \in SCONNECTED$ . Напомним, что, вычисляя  $f$  мы можем выводить сколько угодно по памяти, а мы будем выводить граф.  $f(G; s; t)$  делает следующее: соединим  $t$  со всеми вершинами, кроме  $s$  и все вершины, кроме  $t$ , соединим с  $s$ . Остальные рёбра — рёбра начального графа. Тогда в этом графе  $G'$  есть путь от  $t$  до  $\forall$  другой вершин. Если в  $G$  теперь есть путь из  $s$  в  $t$ , то из  $\forall$  вершин мы можем перейти в  $s$ , а из  $s$  до  $t$ , а оттуда до  $\forall$  другой вершин  $\Rightarrow G'$  сильно связан. Если же в  $G$  нет пути из  $s$  в  $t$ , то и в  $G'$  его не будет, т.к. мы добавили только рёбра, входящие в  $s$  и исходящие из  $t$ .

Таким образом:  $(G; s; t) \in \text{DPATH} \Leftrightarrow f(G; s; t) \in \text{SCONNECTED}$ .

Поэтому это вычислимо на логарифмической памяти? Пусть граф  $G$  дан в виде последовательности рёбер. Тогда надо перенести  $G$ , что не требует доп. памяти, кроме сохранения индекса, где мы сейчас находимся. А далее перебор всех вершин, кроме  $s$  и  $t$ , требует сохранения константного числа вершин и позиций, и каждый шаг просто выводим нужные новые рёбра, и получаем  $G'$  на выходной ленте.  $\Rightarrow$

$\Rightarrow \text{DPATH} \leq \text{SCONNECTED} \Rightarrow \text{SCONNECTED}$  - NL-трудный.

Т.к.  $\text{SCONNECTED} \in \text{NL}$  и NL-трудный  $\Rightarrow \text{SCONNECTED} \in \text{NL}$  ■

③  $\text{CYCLE} = \{G \mid \text{в ориентированном графе } G \text{ есть цикл}\}$

Теорема  $\text{CYCLE} \in \text{NL}$

□ 1) Сначала докажем, что  $\text{CYCLE} \in \text{NL}$ . Решим на логарифмической памяти на НМТ.

Переберём все рёбра  $u \rightarrow v$ . Проверим, есть ли в  $G$  путь от  $v$  до  $u$ . Это можно сделать на логарифмической памяти на НМТ (задача  $\text{DPATH}$ ).  $\Rightarrow \text{CYCLE} \in \text{NL}$

2) Докажем, что  $\text{CYCLE}$  - NL-трудный. Хотим  $\text{DPATH} \leq \text{CYCLE}$ . Для этого по

$(G; s; t)$  надо построить  $G'$ .  $G'$  будем выводить сразу на ленту.  $V = \{s; v_1; v_2; \dots; v_n; t\}$ .

В графе  $G'$  будет  $n+2$  слоя, и  $(n+1)^2$  рёбер. Обозначим для удобства вершины:

$s; i_1; i_2; \dots; i_n; v_1; i_1; v_1; i_2; \dots; v_1; i_n; v_1; t$   $\square$  В графе  $G$  есть ребро  $u \rightarrow v$ . Тогда в  $G'$

будут рёбра  $u_1 \rightarrow v_2; u_2 \rightarrow v_3; \dots; u_n \rightarrow v_{n+1}$ . Так мы строим для всех рёбер, это

не требует доп. памяти, кроме хранения информации, какое мы сейчас рассматриваем

ребро. Просто выводим  $n-1$  ребро нового графа. Потом добавляем рёбра

$t_1 \rightarrow t_n; t_2 \rightarrow t_n \dots t_{n-1} \rightarrow t_n$ . а ребро  $t_n \rightarrow S_1$ . Докажем, что в  $G$  есть путь от  $s$  до  $t \Leftrightarrow$  в  $G'$  есть цикл. Если в  $G$  есть путь от  $s$  до  $t$ , то от  $S_1$  мы дойдем за  $k$  шагов до  $t_k$ . Из  $t_k$  до  $t_{n-1}$ , из  $t_{n-1}$  до  $S_1$  - наш цикл.

Если нет пути от  $s$  до  $t$ , то от  $S_1$  мы не дойдем до какого-либо  $t_k \Rightarrow$

не дойдем до  $t_{n-1}$ , но каждое ребро гарантированно спускает нас на 1 уровень ниже, но единственное ребро вверх - это  $t_{n-1} \rightarrow S_1 \Rightarrow$  цикла не будет  $\Rightarrow$  ДРАТИ  $\leq_L$  CYCLE  $\Rightarrow$  CYCLE - NL-трудный

Т.к. CYCLE  $\in$  NL и CYCLE - NL-трудный  $\Rightarrow$  CYCLE  $\in$  NLC

Т.к. CYCLE  $\in$  NL и CYCLE - NL-трудный  $\Rightarrow$  CYCLE  $\in$  NLC

### Теорема (эквивалентность определений NL)

$NL = NL_{\text{серт}}$ , где  $NL$  - класс, задаваемый как  $NL = NSPACE(\log n)$ , а

$NL_{\text{серт}}$  - класс задаваемый сертификатным определением.

□ 1)  $NL \subseteq NL_{\text{серт}}$ .  $\exists$  НМТМ, распознающая язык  $L$  на логарифмической памяти.  $\Rightarrow$  есть принимающая ветка. Запишем в строку выборы всех недетерминированных переходов.  $\exists$  ДМТ  $V(x;s)$  примет в качестве  $s$  эту строку и будет эмулировать одну конкретную ветку НМТ.  $M$  на каждой ветке тратит память не более, чем логарифмическую  $\Rightarrow V$  работает на логарифмической памяти.

Если принимающей ветки нет, то и сертификата не будет  $\Rightarrow NL \subseteq NL_{\text{серт}}$ .

2)  $NL_{\text{серт}} \subseteq NL$ .  $\exists$  Есть  $V(x;s)$ . Построим НМТ  $M$ . Она будет эмулировать работу  $V$ , но каждый раз при обращении  $V$  к ленте  $s$ , будет угадывать недетерминиро-

важный переход к следующему биту. Возможность чтения с единички гаранти-  
рует, что недеривированный выбор происходит столько раз, сколько битов в  
сертификате, что даёт построить НМТ, если зафиксировать длину сертификата ■

$\overline{\text{PATH}} = \{G; s; t\} \mid \text{в ориентированном графе } G \text{ нет пути из } s \text{ в } t\}$

Теорема (Иммермана - Селенгем)

$\overline{\text{PATH}} \in NL$

□ Нам дан граф  $G$  и вершины  $s$  и  $t$ . Предоставим сертификат, показывающий,  
что из  $s$  в  $t$  нет пути. Введём обозначение:  $N_i(s)$  - множество вершин  
на расстоянии  $\leq i$  от вершины  $s$ . Тогда нас интересует вопрос о принадлежности  
вершины  $t$  множеству  $N_n(s)$ , где  $n = |V| - 1$ .

1) Рассмотрим задачу о принадлежности вершины  $x$  множеству  $N_i(s)$ . Такая задача,

2) лежит в  $NL$ , т.к. сертификатом является последовательность вершин, в которые  
нужно переходить. А верификатор на каждом шаге проверяет, что  $\exists$  ребро в  
следующую вершину (храним только вершину в которой находимся). Также на каждом  
шаге увеличиваем счётчик шагов и проверяем в конце, что он  $\leq i$ .

3)  $|N_i(s)| = m$ . Покажем, что задача проверки, что вершина  $t$  не принадлежит  
множеству  $N_i(s)$  лежит в  $NL$ . Передадим в качестве подсказки множество вершин  
внутри  $N_i(s)$  в возрастающем порядке, а после каждой вершины дополнительно  
передадим сертификат того, что  $v \in N_i(s)$  (из п.1). Тогда алгоритм может

проверить, что каждая из переданных вершин действительно  $\in N_i(s)$ , а возрастающий порядок даёт проверить, что все вершины различны. Также алгоритм проверяет, что все переданные вершины отличны от  $t$  и считает их количество. Если передано ровно  $m$  вершин, и все они не совпадают с  $t$ , то  $t \in N_i(s)$ .

3) Покажем, что зная, что  $|N_{i-1}(s)| = m$  можно показать, что вершина  $t$  не лежит во множестве  $N_i(s)$ . Делается это похоже на шаг 2: передадим в качестве подсказки все вершины из множества  $N_{i-1}(s)$ . Тогда для каждой вершины необходимо дополнительно проверить, что ~~не переданы вершины~~ из какой вершины нет ребра в  $t$ . Это делается,  $h$ , с использованием логарифмической памяти.

4) Покажем, что задача проверки, что  $|N_{i+1}(s)| = k$  при известной мощности  $|N_i(s)| = m$  тоже лежит в  $NL$ . Мы покажем, что если  $x \in N_{i+1}(s)$ , то  $\exists$  доказывающий это сертификат (н.1). И если  $x \notin N_{i+1}(s)$ , то тоже  $\exists$  доказывающий это сертификат (н.3). Тогда передадим в качестве сертификата последовательно сертификаты для всех вершин, показывающих, принадлежат ли они  $N_{i+1}(s)$ . Тогда алгоритм должен проверить все сертификаты и посчитать число подходящих вершин.

5) Осталось соединить всё вместе и описать сертификат. Этого, он состоит из  $n$  уровней. На каждом уровне <sup>передается</sup> ~~определяется~~ размер отсечного множества  $N_i(s)$ , потом он проверяется. Далее передаём размер ~~каждого~~ текущего множества и

сертификат для каждой вершины, принадлежит ли она ему. Внутренние сертификаты описаны в п.1 и п.2. В самом конце передаётся сертификат из п.2, доказывающий, что  $t \in N_n(s)$ . Заметим, что для всех проверок на текущем шаге  $i$  требуется знать только  $|N_{i-1}(s)|$ , что занимает также логарифмическую память.

Таким образом,  $\overline{\text{DPATH}} \in NL$  ■

Теорема  $NL = coNL$

□ В силу того, что  $\text{DPATH} \in NL \Rightarrow \overline{\text{DPATH}} \in coNL$ . По теореме Immerman-Szelepcsényi  $\overline{\text{DPATH}} \in NL \Rightarrow NL = coNL$  ■